SafetyWatch interview prep

Everything here is taken from the code in this repository (and from the installed Better Auth 1.7.7 source where the answer depends on the library's defaults). Answers are written the way you would say them out loud. Links point at the file that proves the answer, so you can re-read it the night before.

Contents

0. Read this first: your resume does not match this repository
1. The pitch
2. Numbers to know, and where each one comes from
3. Architecture and the path of one report
4. Technical questions by area
5. Behavioural questions and stories
6. The awkward questions
7. System design follow-ups
8. Concepts to explain from scratch
9. Before the interview

---

0. Read this first: your resume does not match this repository

The resume at `Portfolio/public/resume.pdf` (also downloadable from your portfolio site) describes an older version of SafetyWatch. An interviewer will read the resume, then open GitHub. Every mismatch below is a question waiting to happen. **Fix the resume before you send it anywhere**, and until then know how to answer.

Resume says	The code says	What to say if asked
"Prisma + **MySQL** backend"	PostgreSQL (Neon) through Prisma 7 with the `pg` driver adapter	"The first version used MySQL. I moved it to Postgres: I needed `AT TIME ZONE` for local-time statistics and wanted Neon's serverless Postgres on Vercel. Prisma made the switch mostly a schema change." Only say the first sentence if it is true; otherwise say the resume is out of date.
"**OCR**-based evidence extraction"	Gemini multimodal model drafts title, description and crime type from a photo, constrained to a JSON schema	It is not OCR. Call it "image analysis with a vision LLM". OCR only reads text; this describes the scene.
"**Real-time** dashboards"	No WebSockets or polling. Pages render on the server per request; mutations call `revalidatePath`	"Dashboards always show current data because they are server-rendered on every request and mutations revalidate them. There is no push; an admin sees another admin's change on the next navigation." Do not claim realtime.

Resume says	The code says	What to say if asked
"SSR/**SSG** via Next.js"	Everything is dynamically server-rendered: the root layout reads the session cookie, which makes every page dynamic. Only `robots.txt` and the sitemap are static.	"Server Components rendered per request. Static generation was not a fit because every page shows the signed-in user in the nav."
"Achieved **sub-1.5s load times**"	Nothing in the repository measures this	Measure it before quoting it (section 9 tells you how), and be ready to say which metric, on which page, on what network. If you cannot, remove it.
"Role-based dashboards and Google OAuth"	True	Covered in 4.3 to 4.5.
"May 2025"	The repository's commits are dated October 2026	Be ready to explain: built earlier, rebuilt and published later.

---

1. The pitch

One sentence

SafetyWatch is a community crime-reporting site: anyone can file a report in three steps, with or without an account, follow it with a tracking ID, and see admin-verified incidents on a list, a map and charts.

Thirty seconds

Reporting an incident should take a couple of minutes, should not require an account, and must never expose the reporter. In SafetyWatch you fill in the details, drop a pin on a map, review and submit, and you get a 16-character tracking ID to check the status later. Admins move each report through a review workflow, and only verified reports become public: a paginated list sortable by distance from you, a Leaflet map with clustering, a heatmap and a satellite layer, and statistics by type, place, month and hour. It is Next.js 16 with Server Components and Server Actions, PostgreSQL through Prisma, Better Auth for email/password and Google sign-in, private Vercel Blob storage for photos, and an optional Gemini feature that drafts the report from a photo.

Two minutes: add these, in order

1. **Privacy is the design driver.** Anonymous reports carry no user id at all. Photos are re-encoded in the browser, which strips EXIF GPS data, and stored in a private blob store whose URLs never reach the browser. Public queries never select the reporter's name or email.
2. **Access rules live on the server.** Every mutation is a server action that re-checks the session and role and validates with Zod. Roles cannot be set by users at all; only a seed script makes an admin.
3. **The review workflow is a state machine** enforced on the server with a compare-and-set update, so two admins cannot apply conflicting transitions.
4. **AI drafts, people decide.** "Extract details from image" pre-fills the form from a photo with structured output validated by the same Zod schema; the reporter still reviews and submits.

5. **Honest limits.** No email sending (so no verification or password reset), anonymous submissions are not rate limited, and there are no automated tests yet.

What is the hard part?

Say: **getting privacy and access control right across three kinds of visitor** (anonymous, signed-in reporter, admin) and four places data leaves the server (pages, the map's popup endpoint, the photo route and the tracking page), without a single helper forgetting a rule. That is why every read of the report table goes through one module, queries.ts, with the public filter in the `WHERE` clause and an explicit `select` on every query.

---

2. Numbers to know, and where each one comes from

None of these are performance benchmarks. They are design limits; you got each one by choosing it, and you should know why.

What	Number	Where it comes from / why
Steps to file a report	3	Details → location → review (report-form.tsx)
Accounts needed to report or track	0	Anonymous reporting + tracking ID
Review states	5	`PENDING, REVIEWING, VERIFIED, RESOLVED, REJECTED` (schema.prisma)
Crime types	7	Theft, burglary, assault, fraud, vandalism, suspicious activity, other
Tracking ID	16 hex characters = 64 random bits	`randomBytes(8).toString("hex")` (actions.ts). 2^64 ≈ 1.8 × 10^19 possibilities, so guessing one is infeasible.
Evidence photos	max 3 per report, each ≤ 1 MB after resizing, longest side ≤ 1600 px	report-meta.ts, image-resize.ts
Server Action body limit	4 MB	next.config.ts. Three 1 MB photos plus fields fit, and it stays under Vercel's 4.5 MB request limit.
Title / description / address	5–120 / 10–5000 / 5–300 characters	validation.ts
Password length	8–128 (users), 12+ (admin via seed)	auth.ts, seed.ts
Session lifetime	7 days, refreshed once a day of use	Better Auth defaults (`expiresIn` 7 d, `updateAge` 1 d)
Sign-in / sign-up rate limit	3 requests per 10 seconds per IP, production only	Better Auth's default special rule; counters in the `rateLimit` table
Page sizes	12 public, 12 "my reports", 20 admin	queries.ts

What	Number	Where it comes from / why
Demo data	64 reports, deterministic	Seeded PRNG with seed `20261011`, about 35% anonymous, spread over the last 240 days (seed.ts)
Nearby sort precision	3 decimal places ≈ 110 m	sort-controls.tsx rounds the position before putting it in the URL
Earth radius in the distance formula	6371 km	Haversine in SQL
Future-date tolerance	5 minutes	Allows clock skew between browser and server
Nominatim timeout	4 seconds	geocoding.ts
Gemini temperature	0.4	Low enough to stay factual, not zero so it can phrase a description

"Sub-1.5 s load time" (on the resume). If you keep it, you need to have measured it. See section 9 for how. Know the vocabulary: **LCP** (Largest Contentful Paint, when the main content is visible; "good" is under 2.5 s), **FCP** (first paint of any content), **TTFB** (time to first byte, dominated here by the server render plus database round trips), **INP** (responsiveness), **CLS** (layout shift).

---

3. Architecture and the path of one report

```
Browser                          Next.js on Vercel (Node runtime)                External
--------                         --------------------------------                --------
Server Components (HTML) <----   app/ pages: read session, query via           PostgreSQL (Neon)
                                 server/reports/queries.ts (Prisma)  --------->  via
@prisma/adapter-pg
Client Components:
  report form, maps, charts
  |  createReport(formData) --> "use server" action:
  |  (Server Action = POST)      1 getSessionUser()  (Better Auth, cookie)
  |                              2 zod-validate fields, parse zoned time
  |                              3 check photos: count, size, JPEG bytes
  |                              4 put() each photo ------------------------->  Vercel Blob
(private)
  |                              5 prisma.report.create (trackingId)  -------->  Postgres
  |                              6 revalidatePath(...)
  |<-- { ok, trackingId } -----
  |  /api/auth/*  ------------->  Better Auth handler  <---------------------->  Google OAuth
  |  /api/reports/:id/summary ->  map popup (public reports only)
  |  /reports/:id/images/:n --->  checks visibility, streams from Blob
  |  Nominatim (from browser) ------------------------------------------------>  OpenStreetMap
  |  Map tiles (from browser) ------------------------------------------------>  OSM / Esri tiles
```

Folder layout to describe:

- `src/app/` routes. Server Components by default, Client Components only where something is interactive.
- `src/server/` server-only code: reports/queries.ts (every read), reports/actions.ts (every write), storage.ts, ai/extract-details.ts, datetime.ts. Most files start with `import "server-only"`, which makes the build fail if a Client Component imports them.
- `src/lib/` shared code: env, auth, session, validation schemas (used by both forms and actions), report metadata and rules.
- `src/components/` UI by feature: `report-form/`, `maps/`, `admin/`, `auth/`, `ui/` (shadcn-style Radix wrappers).

The path of one report, out loud

1. The visitor opens `/reports/new`. The Server Component reads the session. Admins see "Admins cannot submit reports". Everyone else gets the form with today's date and time in the app's time zone pre-filled.
2. **Details step.** React Hook Form with the Zod resolver validates as you type. Photos are downscaled in the browser. Optionally "Extract Details from Image" sends one photo to a server action that asks Gemini to draft the fields.
3. **Location step.** The browser's geolocation (if allowed) drops a pin, or you click the map, or type an address. A pin triggers reverse geocoding (Nominatim) to fill the address; a typed address triggers forward geocoding to move the pin.
4. **Review step** shows everything, including who you are reporting as (or "anonymous").
5. Submit builds a `FormData` and calls the `createReport` server action. It re-checks the session, refuses admins, validates every field again with Zod, converts the local date and time into an instant, checks the photos' bytes, uploads photos to the private blob store, inserts the row with a fresh tracking ID, and revalidates the dashboards. If the insert fails it deletes the uploaded photos.
6. The form swaps to a success screen showing the tracking ID and a link to its tracking page.
7. An admin sees it on `/admin/dashboard` as Pending, moves it to Reviewing, then Verified (public) or Rejected (archived), and later Resolved.

---

4. Technical questions by area

4.1 Next.js App Router, Server Components and Server Actions

Why Next.js? One codebase for the UI and the server logic, Server Components so most pages ship no JavaScript for data fetching, Server Actions so forms call typed server functions without a hand-written API, and first-class deployment on Vercel.

What is a Server Component? A React component that runs only on the server. It can be `async`, read cookies, and query the database directly; its output is sent as a serialised tree and HTML, and its code never reaches the browser. Here pages like crimes/page.tsx call `listPublicReports()` directly.

And a Client Component? A file with `"use client"`. It is server-rendered once to HTML and then hydrated in the browser, so it can use state, effects and browser APIs. The report form, the maps, the charts and the admin status dropdown are Client Components. The rule I followed: keep data fetching and access checks in Server Components and server actions; make the leaves interactive.

What is a Server Action and how is it called? An `async` function in a `"use server"` file. When a Client Component calls it, Next.js sends a `POST` to the current page with a `Next-Action` header that identifies the function, and the arguments serialised in the body (here a `FormData`). The return value comes back serialised. So `createReport(formData)` in the browser is really an HTTP request.

So are Server Actions secure by default? No, and that is the key point. **They are public HTTP endpoints.** Anyone can call them with any arguments. That is why every action in actions.ts re-checks the session and role and validates everything with Zod before touching the database, even though the UI would never send bad data. Next.js does help with CSRF: it compares the request's `Origin` header with the host and rejects mismatches for Server Actions.

How do pages stay fresh after a change? After a mutation the action calls `revalidatePath` for every page that shows that report (dashboards, lists, map, stats, the report page). In practice these pages are rendered per request anyway, because the root layout reads the session cookie, so `revalidatePath` mostly clears the client-side router cache so a back-navigation does not show stale data.

Why is every page dynamic? layout.tsx calls `getSessionUser()`, which reads request headers to show the right navigation. Reading headers or cookies opts the route into dynamic rendering. For this app that is correct: the nav, the dashboards and visibility all depend on who is asking.

What does `cache()` do in `getSessionUser`? React's `cache` memoises the function for the duration of one server request. The layout, the page and any action in the same request share one session lookup instead of three database queries. It does not cache across requests, so a role change takes effect on the next request.

Why `dynamic(() => import(...), { ssr: false })` for the maps? Leaflet touches `window` and `document` the moment it is imported, which crashes during server rendering. `ssr: false` only renders it in the browser. In Next 16 `ssr: false` is only allowed inside a Client Component, which is why there are small loader files like crime-map-loader.tsx that a Server Component page can import.

How is the environment validated? env.ts parses every server variable with Zod (a Postgres URL, a 32+ character secret, the Google and Gemini keys, a valid IANA time zone). instrumentation.ts imports it when the server starts and calls `process.exit(1)` on failure, so a missing key is a startup failure, not a 500 on the first request that needs it. There are no fallback values for secrets. None of these are `NEXT_PUBLIC_`, so none can leak into the client bundle.

4.2 The data schema

Six tables (schema.prisma). Four belong to Better Auth, one is for rate limiting, one is the app's.

Table	Columns that matter	Notes
`user`	`id uuid`, `name`, `email UNIQUE`, `emailVerified`, `image`, `role (USER/ADMIN)`, timestamps	Better Auth's shape plus our `role`. `role` is declared to Better Auth with `input: false`, so no endpoint can set it.
`session`	`id`, `token UNIQUE`, `expiresAt`, `ipAddress`, `userAgent`, `userId → user CASCADE`	One row per signed-in browser. Index on `userId`.

Table	Columns that matter	Notes
`account`	`providerId` (`credential` or `google`), `accountId`, `userId`, `password` (scrypt hash for `credential`), `accessToken`, `refreshToken`, `idToken`, expiry fields, `scope`	`UNIQUE(providerId, accountId)`. A user can have a password account and a Google account linked to the same `user`.
`verification`	`identifier`, `value`, `expiresAt`	Used by Better Auth for OAuth state (and would be for email verification).
`rateLimit`	`key UNIQUE`, `count`, `lastRequest bigint`	Rate-limit counters in the database, because in-memory counters do not survive between serverless invocations.
`report`	see below	

`report` columns: `id uuid`, `trackingId char(16) UNIQUE`, `title varchar(120)`, `description text`, `crimeType enum`, `status enum default PENDING`, `location varchar(300)` (the address), `city varchar(80)?` (for statistics), `latitude float`, `longitude float`, `incidentAt timestamptz(3)`, `isAnonymous boolean`, `imageUrls text[]` (private blob URLs), `actionDetails text?` ("Investigator Notes"), `userId → user SET NULL`, `createdAt`, `updatedAt`.

Indexes, each tied to a query:

- `(status, incidentAt DESC)`: public list, map, stats ("verified, newest first").
- `(userId, createdAt DESC)`: "my reports".
- `(incidentAt DESC)` and `(crimeType)`: admin table sorts.
- `(latitude, longitude)`: intended for bounding-box map queries. (Be honest: the "nearest" query computes the distance for every public row, so it does not use this index. See 4.9.)
- `(city)`: "top locations" statistic.

Why is the `CrimeType` enum declared alphabetically? Postgres sorts enum values in declaration order, not alphabetically. Declaring them A to Z means `ORDER BY "crimeType"` in the admin table sorts by name. `ReportStatus` is declared in workflow order on purpose, so sorting by status follows the workflow.

Why `timestamptz` for `incidentAt`? It stores an instant. The app converts the reporter's wall-clock input in `APP_TIME_ZONE` to an instant on the way in, and back on the way out, so the report reads the same for everyone and "by hour" statistics mean local hours.

Why are photo URLs an array column and not a table? At most three, always read and deleted together with the report, never queried by themselves. A separate table would add a join for nothing. If photos gained their own metadata (captions, moderation), they would get a table.

What happens when a user is deleted? Sessions and accounts cascade. Reports keep existing with `userId = NULL` (`onDelete: SetNull`), because a crime report is community data, not the user's private data. (There is no account-deletion feature yet.)

Why Prisma? Type-safe queries generated from the schema, migrations as SQL files in the repo, and a readable `select` that makes "which columns leave the database" explicit on every query, which is exactly the property the privacy rules need. Where Prisma's query builder cannot express something (distance sort, `AT TIME ZONE` grouping) I use `prisma.$queryRaw` with tagged templates, which parameterise every interpolated value, so it is not open to SQL injection.

Prisma 7 specifics you may be asked about. The new `prisma-client` generator writes the client into `src/generated/prisma` (git-ignored, regenerated on `postinstall`). The database URL lives in prisma.config.ts, not the schema. The app connects through a **driver adapter** (`@prisma/adapter-pg`, the `pg` library) instead of Prisma's Rust engine. prisma.ts keeps one client on `globalThis` in development so hot reloads do not open a new pool each time.

Why two database URLs? Neon gives a pooled URL (through PgBouncer in transaction mode) and a direct one. The app uses the pooled one, which is what you want for many short-lived serverless connections. Migrations use the direct one (`DATABASE_URL_UNPOOLED`), because migrations need session-level features such as advisory locks that a transaction-mode pooler does not support.

4.3 Authentication: what happens to a password

Which library and why? Better Auth. It provides email/password and Google sign-in with sessions stored in my own Postgres through the Prisma adapter, it is TypeScript-first, and it lets me add a server-owned `role` field. Writing auth by hand is where most security bugs come from; I would rather configure a well-reviewed library and spend my effort on the authorisation rules, which are specific to this app.

Walk me through sign-up.

1. register-form.tsx validates with Zod on the client: name 2–80, valid email, password 8–128, and the two password fields match. This is only for quick feedback.
2. `authClient.signUp.email({ name, email, password })` posts JSON to `/api/auth/sign-up/email`. **The password travels in plain text in the request body, protected by HTTPS.** It is never in a URL, never stored in the browser, never logged. Note that the role is not sent, and would be ignored if it were.
3. Better Auth's route handler (app/api/auth/[...all]/route.ts) checks the `Origin` header against the trusted base URL, applies the rate limit, validates length 8–128 on the server, and rejects an existing email.
4. The password is hashed with **scrypt**: a random 16-byte salt, parameters N = 16384, r = 16, p = 1, a 64-byte key, stored as `salt:key` in hex in `account.password` (with `providerId = 'credential'`). The password is NFKC-normalised first, so visually identical Unicode strings hash the same.
5. A `user` row and a `credential` `account` row are created, a session is created, and the cookie is set. The user lands on the home page signed in.

Walk me through sign-in. `authClient.signIn.email({ email, password })` → `/api/auth/sign-in/email`. Better Auth finds the user and their credential account and verifies the scrypt hash. If the user does not exist, it **still hashes the submitted password**, so the response takes the same time and an attacker cannot tell which emails are registered by timing. Same error for both cases ("Invalid email or password", 401). Rate limit: 3 attempts per 10 seconds per IP in production, counted in the database. The client clears the password field after a failure.

Why scrypt and not SHA-256? Why not bcrypt? Passwords have little entropy, so an attacker with a leaked database will try to guess them. SHA-256 is designed to be fast, billions per second on a GPU. scrypt is deliberately slow and **memory-hard** (each guess needs about 32 MB here: 128 × N × r bytes), which makes GPU and ASIC attacks much more expensive. bcrypt is also fine; it is slow but not memory-hard. Argon2id is the current recommendation. Better Auth defaults to scrypt because Node has it built in.

Why not hash the password in the browser? Then the hash becomes the password: whoever captures it can replay it. TLS already protects it in transit, and the server has to do a salted slow hash anyway.

How is the session kept?

- On sign-in Better Auth inserts a `session` row with a random 32-character token, the IP and the user agent, expiring in 7 days.
- It sets a cookie `better-auth.session_token` (with the `__Secure-` prefix over HTTPS) whose value is the token plus an **HMAC signature** made with `BETTER_AUTH_SECRET`. Attributes: `HttpOnly` (JavaScript cannot read it, so an XSS cannot steal it), `SameSite=Lax` (not sent on cross-site POSTs), `Secure` in production, `Path=/`.
- On each request, session.ts calls `auth.api.getSession({ headers })`. Better Auth verifies the signature, looks the token up, checks expiry, and if the session is more than a day old, extends it (sliding expiry).
- The browser never holds a token the app can read. No JWT, no `localStorage`.
- Sign-out (`authClient.signOut()`) deletes the session row and clears the cookie.

Why sessions and not JWTs? A database session can be revoked immediately: sign-out or an admin action deletes the row. A JWT is valid until it expires unless you add a denylist, at which point you are back to a database lookup. The seed script relies on this: when it sets the admin's password it also deletes all of that user's sessions, signing out anyone else who held the account.

Is the session token hashed in the database? No. Better Auth stores it as is; the cookie is signed, but a leaked `session` table would contain usable tokens until they expire. Contrast with my other project, SynChat, where I store only a SHA-256 of the token. If asked how to harden: Better Auth supports a secondary storage and hooks; or shorten the lifetime; or rotate the secret, which invalidates every signature.

What does the role check look like? `getSessionUser()` returns `{ id, name, email, role }`, loaded from the database on every request, so promoting or demoting someone takes effect immediately. Pages use `requireUser()` (redirects to `/login`) or `requireAdmin()` (redirects home). Actions check `viewer?.role === "ADMIN"` themselves, because a redirect helper is for pages, not for an endpoint.

How does anyone become an admin? Only through `npm run db:seed` with `ADMIN_EMAIL` set (seed.ts). `role` is `input: false` in the Better Auth config, so sign-up, Google profile data and the update-user endpoint all ignore it. The seed handles three cases carefully: a new admin (created with `emailVerified: true`, plus a password if `ADMIN_PASSWORD` is set), an existing account with a password given (sets the password, deletes every session, promotes), and an existing **unverified** account with no password given, which it refuses to promote, because anyone could have registered that address.

What is CSRF and how is it handled here? Cross-site request forgery: another site makes your browser send a request with your cookies. Three layers: the cookie is `SameSite=Lax`, Better Auth checks the `Origin` header of its POST endpoints against the trusted origin, and Next.js checks `Origin` against `Host` for Server Actions.

What is missing? No email sending, so no email verification, password reset or "forgot password". Sign-up reveals whether an email is already registered. Anonymous report submission and the Gemini action are not rate limited. Say it before they find it.

4.4 Google OAuth, step by step

This is the most likely deep question. Learn the sequence.

Set-up. In Google Cloud Console I created an OAuth client (web application), got a client id and secret, and registered the redirect URI `https://safetywatch.arvin.life/api/auth/callback/google` (and the localhost one for development). The id and secret are server environment variables. In auth.ts that is the whole configuration: `socialProviders: { google: { clientId, clientSecret } }`.

The flow (Authorization Code with PKCE):

1. The user clicks "Sign in with Google". login-form.tsx calls `authClient.signIn.social({ provider: "google", callbackURL: "/", errorCallbackURL: "/login" })`.
2. That POSTs to `/api/auth/sign-in/social`. Better Auth generates a random `state` (32 characters) and a **PKCE** `code_verifier`, stores them (with the callback URLs and an expiry) in the `verification` table, and builds Google's authorisation URL containing: `client_id`, `redirect_uri`, `response_type=code`, `scope=openid email profile`, `state`, and `code_challenge` = base64url(SHA-256(code_verifier)) with `code_challenge_method=S256`.
3. The browser is redirected to Google. The user signs in to Google and consents. **SafetyWatch never sees the Google password.**
4. Google redirects back to `/api/auth/callback/google?code=...&state=...`.
5. Better Auth looks up the `state`. If it is unknown or expired, the request is rejected: that is what stops an attacker from completing a login in your browser with their own code (login CSRF).
6. Server to server, Better Auth POSTs to Google's token endpoint with the `code`, the client secret and the original `code_verifier`. Google checks that SHA-256 of the verifier matches the challenge from step 2, so a stolen `code` is useless without the verifier. Google returns an **access token**, an **ID token** (a signed JWT with the user's id, email, `email_verified`, name and picture) and, only if offline access were requested (it is not here), a refresh token.
7. Better Auth reads the profile and decides:
    - **Known Google account** (`account` row with `providerId='google'` and this Google user id): sign that user in.
    - **New email**: create a `user` (with `emailVerified` from Google) and a `google` `account` row.
    - **Email already exists with a password account**: link only if allowed. Better Auth's default `requireLocalEmailVerified: true` means it links only if the existing local account's email is verified. Password sign-ups here are never verified (no email sending), so linking is refused and the user is sent to `/login?error=account_not_linked`, which login/page.tsx turns into "An account with this email already exists. Sign in with your password instead."
8. A session row is created, the session cookie is set, and the browser is redirected to `callbackURL` (`/`).

Why refuse to link to an unverified account? Account pre-hijacking. Without that rule, an attacker could register `victim@gmail.com` with a password of their choosing before the victim ever visits. When the victim later signs in with Google, the accounts would merge, and the attacker's password would still open the victim's account. Requiring the local email to be verified closes that.

Why does the admin seed set `emailVerified: true`? The operator vouches for `ADMIN_EMAIL`. That is what allows the admin to sign in with Google using that address and have it link to the admin row.

What is the difference between the access token and the ID token? The **ID token** answers "who is this user" (OpenID Connect): a signed JWT for the client. The **access token** authorises calls to Google APIs on

the user's behalf. SafetyWatch only needs identity, so it only needs the ID token's claims.

Where are Google's tokens stored, and is that a risk? In the `account` table (`accessToken`, `refreshToken`, `idToken`), in plain text by default. The app never uses them after sign-in. Better Auth has an `account.encryptOAuthTokens` option; turning it on, or not persisting them, would be the hardening step. Good to volunteer.

What is PKCE and why use it when there is a client secret? Proof Key for Code Exchange: the client proves at token-exchange time that it is the same party that started the flow. It was designed for apps that cannot keep a secret (mobile, SPAs), but the current OAuth security best practice recommends it for every client, because it also defends against authorisation-code injection.

What is `state` for? CSRF protection for the OAuth redirect. It ties the callback to the browser session that started the flow.

Why is the scope `openid email profile` and not more? Least privilege: the app only needs to know who you are.

4.5 Authorisation and privacy rules

Have these as a list; interviewers like a crisp rule set.

1. **Public visitors** only ever see reports whose status is `VERIFIED` or `RESOLVED`. The filter is in the SQL `WHERE` clause (`publicWhere`), never applied after fetching.
2. **Public queries never select the reporter.** Every query has an explicit `select`; only admin queries join `user` for name and email.
3. **Anonymous reports have no `userId` at all.** Not a hidden flag: the column is `NULL`. So no join, bug or admin screen can reveal who filed it. A signed-in user who ticks "anonymous" also gets `userId = NULL` (they then follow it by tracking ID, not in their dashboard).
4. **Owners** can see their own report in any status and edit it only while `PENDING`. The update is `updateMany({ where: { id, userId: viewer.id, status: "PENDING" } })`: ownership and status are part of the `WHERE`, so there is no gap between checking and writing.
5. **Admins** see everything, change status (only along the workflow) and delete. They cannot file reports, so the reviewer and the reporter are never the same account.
6. **Tracking ID lookups** return the title, description, status, location and time of that one report, and a link to the full page only if the viewer is allowed to see it. No reporter details, coordinates or photos. Tracking pages are `noindex` and disallowed in `robots.txt`.
7. **Photos** are served through images route, which applies the same visibility rule as the report page before streaming the file.
8. **The map popup endpoint** (summary route) only returns public reports.
9. A non-visible report and a non-existent one both return 404 / "not found", so ids cannot be probed.

Why one queries module? So that "can this person see this row" is decided in one place, next to the query. If every page wrote its own Prisma query, one would eventually forget the status filter.

4.6 The Leaflet map, in depth

Files: crime-map.tsx, location-picker.tsx, report-location-map.tsx, marker-icon.ts.

What is Leaflet and what is React Leaflet? Leaflet is a small JavaScript library that draws an interactive "slippy" map: it requests 256 × 256 px image tiles for the visible area and zoom level from a tile server, positions them, and handles panning, zooming, markers and popups. React Leaflet wraps it in components (`MapContainer`, `TileLayer`, `Marker`, `Popup`) and hooks (`useMap`, `useMapEvents`) so the map can be described declaratively.

Where do the tiles come from? Standard mode: OpenStreetMap's tile servers, URL template `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png` (zoom, column, row). Satellite mode: Esri World Imagery. Both require attribution, which is shown in the corner. The browser fetches tiles directly; they do not pass through my server.

How does the crime map get its data? The `/reports/map` page is a Server Component. It calls `getMapPoints()`, which returns only `{ id, lat, lng, type }` for every public report, and passes that small array as props to the client map. Details are not sent up front: when you click a marker, the popup fetches `/api/reports/:id/summary` (title, type, formatted time, a short excerpt). That keeps the initial payload tiny even with thousands of points, and means report text is only loaded for what you look at.

How does the crime-type filter work? Client-side. `selectedType` is state; `filteredPoints` is a `useMemo` that filters the points array; every layer (heatmap, clusters, markers) renders from `filteredPoints`. The filter buttons are generated only for types that actually occur in the data. Changing the filter also closes any open popup. No server round trip, because all the points are already in the browser.

How does clustering work? `leaflet.markercluster`. All markers are added to a `MarkerClusterGroup`, which groups markers within 60 pixels of each other at the current zoom into one bubble showing the count. Zooming in splits clusters; at the maximum zoom, markers that are still on top of each other "spiderfy" (fan out) when clicked. The cluster bubble is a custom `divIcon` with the count.

How does the heatmap work? `leaflet.heat`. Each point becomes `[lat, lng, intensity=1]`. The plugin draws a blurred circle (radius 40 px, blur 30 px) for each point on a canvas and maps the accumulated intensity to a colour gradient from blue to dark red. Dense areas glow. Heatmap and clusters are mutually exclusive: turning one on turns the other off, and the heatmap closes any popup since there are no markers to anchor it.

Neither plugin has a React component. How did you integrate them? Small components that render `null` and use the `useMap()` hook to get the Leaflet map instance, then in a `useEffect` create the layer imperatively, add it, and **return a cleanup that removes it**. When the points, the marker size or the filter change, React runs the cleanup and the effect again, so the layer is rebuilt from the new data and nothing leaks. That is the standard pattern for wrapping an imperative library in React.

How are the coloured markers drawn? `L.divIcon` with inline HTML: a circle in the crime type's colour with a white pin SVG, sized by the "Marker size" slider (20–50 px). The anchor is the bottom-centre so the tip of the pin is on the location. **Only numbers and fixed colours are interpolated into that HTML, never report text**, because a `divIcon`'s HTML is inserted as markup; putting a user-written title there would be an XSS hole. Report text goes into the React popup, which escapes it.

How is the map centred? Your position if you allow geolocation, otherwise the first report, otherwise the centre of India. A subtle point: `MapContainer`'s `center` prop is only read once, at mount; changing it later does nothing. So there is a `RecenterMap` helper component that calls `map.setView` in an effect when the computed centre changes (for example, when geolocation answers after the map has mounted).

How do popups work? Clicking a marker sets `selected`. A React Leaflet `<Popup>` keyed by the report id is rendered at that position, offset upwards by the marker size. Its content component fetches the summary with an `AbortController` (so a quickly closed popup cancels its request) and shows Loading / error / content. Because the popup was laid out around "Loading…", it calls `popup.update()` once the content arrives so Leaflet re-measures and re-pans. Closing the popup clears the selection, unless another marker was selected in the meantime.

Why are the default marker images in `/public`? Leaflet's default icon references image files by URLs computed relative to its CSS, which break under bundlers like webpack and Turbopack. Serving `marker-icon.png`, the 2× version and the shadow from `/public` and building `L.Icon` with those URLs fixes it.

Why do the overlay controls have `z-index: 1000`? Leaflet's panes use z-indices up to about 700 (popups) and 1000 for controls. Overlays above the map need at least that.

How does the location picker work? A `MapContainer` with `useMapEvents({ click })` that reports the clicked coordinates. When the pin moves from outside the map (geolocation or a typed address), a `FollowPosition` helper calls `map.flyTo` to animate there, zoomed to at least 16. The read-only map on the report page has `scrollWheelZoom={false}` so scrolling the page does not zoom the map.

How does address lookup work? OpenStreetMap's **Nominatim** service, called from the browser (geocoding.ts):

- **Reverse geocoding** (pin → address): `/reverse?format=jsonv2&lat&lon&zoom=16&addressdetails=1`. The display name fills the address field, and the city is taken from the first of `city, town, village, municipality, county, state_district` that exists.
- **Forward geocoding** (typed address → pin): on blur of the address field, `/search?q=...&limit=1`.
- Both have a 4-second timeout, validate the JSON with Zod, and return `null` on any failure: geocoding is a convenience and must never block a report.
- **Race condition:** if you click twice quickly, two lookups are in flight and can answer in the wrong order. A `lookupId` ref is incremented per lookup, and a result is only applied if its id is still the latest.
- While the lookup runs, the address field shows a placeholder like "Location at 19.2183, 72.9781" so it is never empty.
- If the user typed an address by hand and no city came from the map, the server takes a best guess from the comma-separated parts.

Limits of Nominatim? Its public usage policy allows about one request per second and asks for an identifying user agent or referrer; heavy use needs a self-hosted instance or a paid geocoder. Listed under known limitations.

4.7 The report form and evidence photos

How is the multi-step form built? One React Hook Form instance owns all fields across the three steps, with `zodResolver(reportFieldsSchema)`, so the same schema the server uses gives instant feedback. Each step is its own component reading the form through `useFormContext`. The step indicator only lets you open the location step once the details are valid, and the review step once both are. Pressing Enter in a single-line input does not submit early, and only the review step submits. Steps animate with Motion's `AnimatePresence`.

How are server-side errors shown? Server actions return `{ ok: false, error, fieldErrors }`. The form copies each field error onto the field with `form.setError` and jumps back to the step that contains it.

How are the photos handled in the browser? image-resize.ts: `createImageBitmap(file)`, draw onto a canvas no larger than 1600 px on its longest side, export as JPEG at quality 0.82, then 0.7, then 0.55, and if it is still over 1 MB, shrink the dimensions by 25% and try again, up to six rounds. Result: three photos always fit in one request. **Side effect I wanted: re-encoding through a canvas drops all EXIF metadata, including the GPS position where the photo was taken**, which matters for an anonymous reporter.

And on the server? At most three files, each at most 1 MB, and each must start with the JPEG magic bytes `FF D8 FF`. The declared content type and file name are ignored because the client controls them. Photos are uploaded to the private blob store under a random folder with a random suffix, then the row is inserted. If anything fails, the uploaded photos are deleted so nothing is orphaned.

Why a private blob store? A public blob URL works for anyone who has it, forever. With a private store the URL is useless without the server's token, and it is never sent to the browser. The page shows `/reports/:id/images/:n` instead, and that route re-applies the visibility rule (public once verified, otherwise owner or admin), then streams the image. It supports `ETag` / `If-None-Match` so the browser can revalidate cheaply with a 304, sets `Cache-Control: private, no-cache` so shared caches never keep it and the browser always re-checks access, and `X-Content-Type-Options: nosniff`.

4.8 The Gemini "Extract Details from Image" feature

What does it do? Sends one evidence photo to Gemini (`gemini-3.5-flash-lite` by default) from a server action, and gets back a draft title, description and crime type that pre-fill the form. The reporter can change anything before submitting.

Why a server action and not a call from the browser? The API key must never reach the browser. Server-side, I also re-check the image (JPEG, ≤ 1 MB) and block admins.

How do you get reliable JSON back? Structured output: `responseMimeType: "application/json"` with `responseJsonSchema` generated from a Zod schema by `z.toJSONSchema`. The crime type is an enum in that schema, so the model can only pick a valid one. Then I parse the answer with the **same Zod schema**, trim and length-cap the strings, and treat any mismatch as a failure. The provider's promise is not trusted on its own.

Prompt injection: what if the photo contains text like "ignore your instructions"? The prompt says to treat any text in the photo as evidence to describe, never as instructions. More importantly, the blast radius is tiny: the model has no tools, no data access, and its output only pre-fills a form that a human reviews before submitting, and that the server validates again on submit.

What if Gemini is down? The action returns "Could not analyse the image. Please fill in the details yourself." Reporting never depends on it.

Is it OCR? No. OCR extracts text. This is a vision-language model that describes the scene. (The resume says OCR; fix that.)

Cost and abuse? It is not rate limited, which is a real gap: anyone could call the action in a loop and spend the API quota. Fix: a per-IP limit (the same database-backed limiter), or require sign-in for the feature.

4.9 Lists, "nearest first", statistics and time zones

How does "Sort by Nearby" work? The browser asks for geolocation. Because the server does the sorting and pagination, the position goes into the URL, rounded to three decimals (about 110 m, so the exact location is not in history or logs): `/crimes?sort=nearby&lat=19.218&lng=72.978`. The page validates those with Zod (`.catch` falls back to the date sort on bad input) and runs a raw SQL query that computes the **haversine** great-circle distance for every public report and orders by it, with a stable tie-break.

Explain the haversine formula. Distance on a sphere from two latitude/longitude pairs: `2R · asin(√(sin²(Δφ/2) + cos φ1 · cos φ2 · sin²(Δλ/2)))`, with R = 6371 km. The `least(1, …)` inside guards `asin` against a value fractionally above 1 from floating-point rounding.

Does that scale? No, and say so: it computes the distance for every public row, then sorts, so it is a full scan of public reports per page. Fine for thousands of rows. For millions: PostGIS with a `geography` column, a GiST index and the `<->` nearest-neighbour operator, which walks the index in distance order. A cheaper intermediate step is a bounding-box prefilter on `(latitude, longitude)`, which the existing index supports.

Why offset pagination? Simple and supports "page N of M". Its costs: `OFFSET` gets slower deep into the list, and rows inserted while you page can shift items between pages. The fix is keyset (cursor) pagination on `(incidentAt, id)`. Every order-by has `id` as a final tie-break so pages are deterministic. Out-of-range page numbers are clamped.

How are the statistics computed? In the database, in parallel with `Promise.all`: total and latest incident (`aggregate`), counts by crime type and top 10 cities (`groupBy`), and counts by month and by hour of day with raw SQL using `"incidentAt" AT TIME ZONE $APP_TIME_ZONE` so "8 pm" means 8 pm in India, not in UTC. The page receives small arrays and Recharts renders pie and bar charts in a Client Component.

How do you handle time zones when someone enters "14:30"? datetime.ts treats the form's date and time as wall-clock time in `APP_TIME_ZONE` and converts to an instant: compute the zone's UTC offset with `Intl.DateTimeFormat.formatToParts`, subtract it, and do a **second pass** because the offset can differ on either side of a daylight-saving change. It also rejects impossible dates such as 31 February by round-tripping the parsed values. And incidents more than 5 minutes in the future are refused (5 minutes allows for clock skew).

4.10 The admin dashboard and the review workflow

How does the workflow work? `STATUS_TRANSITIONS` in report-meta.ts: Pending → Reviewing → Verified or Rejected; Verified → Resolved; Resolved and Rejected are final. The dropdown only offers valid next states, and the server enforces the same table.

What if two admins change the same report at once? `updateReportStatus` reads the current status, checks the transition, then updates with `WHERE id = ? AND status = <the status I read>`. That is a **compare-and-set** (optimistic concurrency): if another admin changed it in between, zero rows match and the second admin is told "This report was just updated by someone else." No locks held.

How do search, filters, sort and pagination work? All state lives in the URL (`q`, `reporter`, `sort`, `dir`, `page`, `apage`), parsed with Zod where every field has a `.catch` default, so a hand-edited URL can never break the page. The server does the filtering, sorting and paging. Search matches the title (case-insensitive

contains), a tracking-ID prefix, and crime-type or status labels that contain the text. Column headers are links that flip the sort direction. Active and archived (resolved/rejected) reports are two separately paginated tables.

Why URL state and not React state? Shareable and bookmarkable, the back button works, it survives a refresh, and the server can render the right page without client JavaScript.

Delete? Admin only. Delete the row first, then the photos. If the blob delete fails, the files are unreachable anyway (nothing references them and the store is private), so it is logged, not surfaced.

"Investigator Notes"? The report page shows `actionDetails` if present. Be aware: nothing in the app writes it yet; only the seed data has it. If asked, say it is the next admin feature (a text field next to the status change).

4.11 Deployment and operations

How is it deployed? Vercel with the Next.js preset. `npm install` runs `prisma generate`. Database on Neon (pooled URL for the app, direct URL for migrations). A private Vercel Blob store connected to the project. Migrations are applied once with `npm run db:deploy`, then `npm run db:seed` creates the admin and demo data. Custom domain `safetywatch.arvin.life`.

What runs where on Vercel? Pages, server actions and route handlers run as serverless functions in the Node runtime. Static assets are served from the CDN. That is why rate-limit counters are in the database: each invocation may be a fresh process.

A production problem you fixed? (prisma.ts) New connections to a distant database failed with `ETIMEDOUT`. Node's "happy eyeballs" gives each resolved address 250 ms to connect before trying the next, and a far-away region needed longer. `setDefaultAutoSelectFamilyAttemptTimeout(1000)` (and the same flag in `start.sh` for the CLI) fixed it. Good story for "a bug that was not in your code" (section 5).

SEO? Metadata with a title template, Open Graph and Twitter cards, an Open Graph image, `robots.ts` that disallows `/admin/`, `/api/`, `/dashboard` and `/track/`, and a sitemap of public pages. Personal pages are `noindex`.

Monitoring? Only logs (`console.error` on failures). I would add error tracking (Sentry), Vercel Analytics / Speed Insights for real-user Web Vitals, and an alert on Gemini error rate.

4.12 Testing

How did you test it? Be honest: **there are no automated tests in this repository.** TypeScript in strict mode, ESLint and Zod validation at every boundary catch a lot, and I tested the flows by hand: anonymous and signed-in reports, every status transition, the visibility rule as each kind of user, Google sign-in including the "account exists" case.

What would you add first?

1. Integration tests for queries.ts and actions.ts against a real Postgres (Vitest): the public filter, owner-only edit, the state machine and the concurrent status update.
2. Unit tests for datetime.ts around daylight-saving changes and impossible dates.
3. Playwright for the three-step form, anonymous vs signed-in, and the admin workflow.

5. Behavioural questions and stories

Situation, action, result, lesson. Under two minutes each. Reword to match what actually happened.

"Tell me about a hard bug."

Database connections timing out only from some places.

- **Situation.** Locally against Neon, new connections sometimes failed with `ETIMEDOUT`, but the database was up.
- **Investigation.** The host resolved to several addresses. Node tries them with "happy eyeballs" and gives each only 250 ms before moving on. A database in a far region took longer than that to accept the TCP connection, so every attempt was abandoned.
- **Fix.** Raise the per-address timeout to one second, both in the app (prisma.ts) and for the Prisma CLI through `NODE_OPTIONS` in start.sh.
- **Lesson.** Read the error's layer: `ETIMEDOUT` from the network stack is not a database bug or a credentials bug.

"Tell me about a security decision."

Making anonymous really anonymous.

- A flag would have been simple: `isAnonymous = true` and keep the `userId`. But then any admin query, bug or future feature could reveal the reporter.
- So an anonymous report never stores a `userId`. On top, photos are re-encoded in the browser to strip EXIF GPS, the private blob store hides photo URLs, and public queries never select reporter fields.
- Trade-off accepted: a signed-in user who files anonymously cannot see that report in their dashboard; they use the tracking ID like everyone else.

"Tell me about a trade-off."

Pick one:

- **Client-side map filtering vs server queries.** I send every public point (id, coordinates, type only) once and filter in the browser: instant filter changes, one request. It stops working at hundreds of thousands of points; then the server should return points for the visible bounding box, or clusters computed in the database.
- **Server-side "nearest" sort needs your location in the URL.** I rounded it to ~110 m to limit what ends up in history and logs.
- **Better Auth vs writing auth.** Less control (for example, session tokens are not hashed at rest), far less risk.

"Tell me about a time you designed for failure."

Geocoding and the Gemini feature are both conveniences on top of the core flow. Both return `null` or a friendly error on any failure, have timeouts, and never block submitting. The form always has a typed-address and fill-it-yourself path.

"What are you most proud of?"

The single queries module with the visibility rule in every `WHERE` clause and an explicit `select` everywhere. It turns "is this private data safe" from a review of every page into a review of one file.

"What would you do differently?"

- Write tests from the start, especially for the access rules.
- Add email (verification, password reset, status notifications) early; its absence shapes several limitations.
- Use PostGIS from day one for location queries.
- Rate-limit the anonymous endpoints before launch.

"How did you decide what to build first?"

The report flow and the access rules first (the product is worthless if reports leak), then the admin workflow, then the public views (list, map, stats), then the AI drafting as an optional extra.

---

6. The awkward questions

"There are no tests." Correct. Say what you rely on instead (strict TypeScript, Zod at every boundary, a single queries module), how you tested by hand, and exactly which tests you would write first (4.12). Do not pretend.

"Your resume says MySQL / OCR / real-time / SSG / sub-1.5 s." See section 0. Fix the resume. If it comes up anyway: "That line is from an earlier version; the current one is Postgres, a vision model rather than OCR, and server-rendered rather than realtime."

"Can anyone spam anonymous reports?" Yes. There is no rate limit or CAPTCHA on anonymous submission. Mitigations: per-IP limits with the existing database limiter, a CAPTCHA such as Cloudflare Turnstile on the anonymous path, and every report starts unpublished anyway, so spam never reaches the public pages.

"Can't someone enumerate tracking IDs?" 64 random bits. At a million guesses a second it would take about 300,000 years on average to hit one specific report. And a hit reveals only that report's own text and status.

"An admin knows who filed a non-anonymous report. Is that a privacy problem?" It is by design: reviewers may need to contact the reporter. Anyone who does not want that files anonymously, which is one click and available without an account.

"Is it truly anonymous? You have server logs." The app stores nothing linking an anonymous report to a person, but the hosting platform logs IP addresses of requests, as every web server does. True anonymity against the operator would need things like Tor. Say this plainly.

"Who made this, you or an AI assistant?" Answer honestly about the tools you used and for what. Then prove ownership by explaining any part in depth; this document exists so you can.

"The live demo has fake data." Yes, 64 deterministic demo reports from the seed script, so the map and statistics have something to show. Real reports would come from real users.

"Why Next.js and not a separate React app and API?" For this app the UI and the data rules are tightly coupled and the API has one consumer. Server Components and actions remove a whole layer (API routes, client fetching, loading states for every page). If a mobile app needed the same data, I would add a proper API.

---

7. System design follow-ups

"A city with 10,000 reports a day. What breaks?"

1. The map sends every public point: switch to bounding-box queries as you pan, or server-side clustering (PostGIS `ST_ClusterDBSCAN`, or a tiles approach).
2. "Nearest" sort: PostGIS + GiST index.
3. Offset pagination: keyset pagination.
4. Statistics computed on every page view: cache them for a few minutes (Next.js `unstable_cache` / `"use cache"` with a revalidation tag) or keep a materialised view refreshed periodically.
5. Admin review becomes the bottleneck: assignment, queues, priorities, duplicate detection (same place and type within an hour).

"How would you notify reporters when the status changes?" Email (Resend or SES) for signed-in users; for anonymous reporters, an optional email at submission, or web push subscribed on the success page. Send it from the status-change action after the update commits, through a queue so a mail outage does not fail the admin's action.

"How would you add realtime to the admin dashboard?" Postgres `LISTEN/NOTIFY` or a managed service (Pusher, Ably, Supabase Realtime) since Vercel functions cannot hold WebSockets, publishing an event when a report is created or changes; the dashboard subscribes and calls `router.refresh()`.

"How would you moderate images?" A moderation pass (the same Gemini call or a dedicated safety API) before an image becomes public, blurring faces and licence plates, and an admin "hide photo" switch.

"Multiple regions or countries?" `APP_TIME_ZONE` becomes per-report (derive from coordinates), crime types become configurable, and statistics group by region.

---

8. Concepts to explain from scratch

Concept	Plain explanation
Server Component	React component that runs only on the server, can query the database directly, and ships no JavaScript for itself.
Server Action	A server function callable from the client; really a POST endpoint. Must check auth and validate input itself.
Hydration	The browser attaching React's event handlers to server-rendered HTML so it becomes interactive.
OAuth 2.0	A protocol for letting an app act or sign in with another service without ever seeing your password there.
OpenID Connect	An identity layer on OAuth: the ID token says who the user is.
Authorization Code flow	The browser gets a short-lived code; the server exchanges it, with its secret, for tokens. Tokens never pass through the browser's URL.
PKCE	The client sends a hash of a secret at the start and the secret itself at the exchange, proving it is the same client.
`state` **parameter**	Random value tying the OAuth callback to the browser that started the flow; prevents login CSRF.
scrypt	A slow, memory-hard password hashing function; each guess costs time and memory.
Salt	Random data mixed into each password hash so identical passwords hash differently and precomputed tables are useless.
HttpOnly / Secure / SameSite	Cookie flags: not readable by JavaScript / HTTPS only / not sent on cross-site requests (Lax allows top-level navigations).
HMAC	A keyed hash; proves a value was produced by someone with the secret. Used to sign the session cookie.
CSRF	Another site makes your browser send an authenticated request. Stopped by SameSite cookies and Origin checks.
XSS	Injected script running in your page. React escapes text by default; the risk is anywhere raw HTML is inserted (Leaflet `divIcon`).
Compare-and-set / optimistic concurrency	Update only if the row still has the value you read; if not, someone else changed it.
Haversine	Great-circle distance between two points on a sphere from their latitudes and longitudes.
Slippy map / tiles	The map is a grid of 256 px images per zoom level, fetched for the visible area.
Geocoding	Address → coordinates (forward) and coordinates → address (reverse).
Marker clustering	Grouping nearby markers into one counted bubble at low zoom.
Heatmap	Each point adds intensity to a blurred area; colour shows density.
EXIF	Metadata in photos, including the camera's GPS position. Stripped here by re-encoding.

Concept	Plain explanation
Magic bytes	The first bytes of a file that identify its real type (`FF D8 FF` for JPEG).
ETag / 304	A version tag for a response; the browser asks "still this version?" and gets an empty 304 if so.
Connection pooler (PgBouncer)	Shares a few database connections among many short-lived clients; transaction mode cannot support session features like advisory locks.
Structured output	Constraining an LLM's answer to a JSON schema, then validating it in code.
Prompt injection	Text in the model's input trying to act as instructions.
Web Vitals (LCP, INP, CLS, TTFB)	Google's user-centred performance metrics: loading, responsiveness, visual stability, server response.

---

9. Before the interview

Fix

- [ ] **Update the resume** (section 0). This is the most important item. Also update the copy in `Portfolio/public/resume.pdf`, which is public.
- [ ] Decide on the "sub-1.5 s" claim. To measure: open the live site in Chrome, DevTools → Lighthouse → Mobile and Desktop, run on `/`, `/crimes` and `/reports/map`, note **LCP** and **TTFB**, run three times and take the median. Or run PageSpeed Insights on the URL, which also shows real-user data if there is enough traffic. Quote it as "LCP X s on desktop, Lighthouse, median of 3". Note that the first request after idle may be a cold start.
- [ ] Optional quick wins that are easy to talk about: rate-limit anonymous submission and the Gemini action; turn on `account.encryptOAuthTokens`; add a first test file.

Run once, so you are quoting what you saw

- [ ] `./start.sh --seed`, then sign in as the admin and as a normal user.
- [ ] File one anonymous report and one signed-in report with a photo; try "Extract Details from Image".
- [ ] Move a report through every status; try an invalid transition from a second tab to see the conflict message.
- [ ] Sign up with a password, then try Google with the same email, and see the `account_not_linked` message.
- [ ] Open the map: filter by type, switch to satellite, toggle heatmap and clusters, open a popup.

Five-minute demo script

1. Home → "Report anonymously" → three steps with a photo and "Extract Details" → note the tracking ID.
2. Track page with that ID: Pending.
3. Admin dashboard: search by the tracking ID prefix, move it to Reviewing, then Verified.
4. Crime map: it is there; filter, heatmap, popup.
5. Statistics page.

Read these files until you can explain every line

1. src/server/reports/queries.ts: `publicWhere`, `getReportForViewer`, `listPublicReportsNear`, `getPublicStats`
2. src/server/reports/actions.ts: `createReport`, `updateReport`, `updateReportStatus`
3. src/lib/auth.ts and src/lib/session.ts
4. src/components/maps/crime-map.tsx
5. src/components/report-form/report-form.tsx and src/lib/image-resize.ts
6. prisma/seed.ts: `seedAdmin`

Questions to ask them

- How do you decide between server-rendered pages and a separate API in your products?
- How do you handle personal data and anonymity requirements in practice?
- What does your testing pyramid look like for access-control rules?